

KIẾN TRÚC MÁY TÍNH

CHƯƠNG 2: KIẾN TRÚC TẬP LỆNH - PHẦN 3

Trình bày các thông tin cơ bản về hiệu suất máy tính, các phương pháp để tính hiệu suất máy tính, các yếu tố ảnh hưởng đến hiệu suất máy tính và giải pháp để nâng cao hiệu suất.

ThS. Phan Đình Duy

Nội dung môn học

- Chương 1: Những khái niệm và công nghệ máy tính (Tuần 1)
- **Chương 2: Kiến trúc bộ lệnh (Tuần 2, 3, 4)**
- Chương 3: Các phép toán số học trong máy tính (Tuần 5, 6)
- Chương 4: Bộ xử lý (Tuần 7, 8)
- Chương 5: Bộ xử lý pipeline (Tuần 9, 10)
- Ôn tập (Tuần 11)

MỤC TIÊU

1. Trình bày được khái niệm về lệnh, tập lệnh
2. Phân biệt được những nhóm lệnh cơ bản, các loại toán hạng của kiến trúc MIPS
3. Thực hiện chuyển đổi lệnh ngôn ngữ cấp cao sang assembly và mã máy và ngược lại
4. Thực hiện lập trình bằng ngôn ngữ assembly cho MIPS

NỘI DUNG

1. Giới thiệu
2. Các phép tính - Các lệnh trong MIPS
3. Toán hạng (add a, b, c)
4. Số có dấu và không dấu
5. Các phép tính Logic
6. Lập trình ASM trên MARS 4.5 với tập lệnh MIPS
- 7. Biểu diễn lệnh**
- 8. Các lệnh điều kiện và nhảy**

NỘI DUNG

7. Biểu diễn lệnh

- Các định dạng lệnh
- Chuyển lệnh ASM sang mã máy
- Chuyển lệnh từ mã máy sang ASM

8. Các lệnh điều kiện và nhảy

- Các lệnh nhảy có điều kiện
- Các lệnh giả
- Các lệnh nhảy không điều kiện

7. Biểu diễn lệnh

Biểu diễn lệnh

- Lệnh được biểu diễn bằng các mã nhị phân (mã máy)
- Định dạng lệnh là một hình thức biểu diễn một lệnh dưới dạng các trường mã nhị phân
- Lệnh của MIPS đều rộng 32 bit
- MIPS có 3 định dạng lệnh:
 - Định dạng lệnh R: Cho các thao tác tuần tự trên thanh ghi
 - Định dạng lệnh I: Cho các thao tác sử dụng số tức thời có giá trị nhỏ và vừa
 - Định dạng lệnh J: Cho các thao tác sử dụng số tức thời có giá trị lớn

Định dạng lệnh - R_type

Các trường lệnh dạng R:

- op (opcode): Mã lệnh
- rs: Địa chỉ toán hạng thanh ghi nguồn thứ nhất
- rt: Địa chỉ toán hạng thanh ghi nguồn thứ hai
- rd: Địa chỉ toán hạng thanh ghi đích
- shamt (shift amount): Lượng dịch (mặc định là 00000 - chỉ được sử dụng cho các lệnh dịch, xoay)
- funct (function code): Mã lệnh mở rộng cho op

op	rs	rt	rd	shamt	funct
6 bit	5 bit	5 bit	5 bit	5 bit	6 bit

Định dạng lệnh - R_type - Ví dụ 1

op	rs	rt	rd	shamt	funct
6 bit	5 bit	5 bit	5 bit	5 bit	6 bit

add \$t0, \$s1, \$s2

op (add)	\$s1	\$s2	\$t0	0	funct (add)
0	17	18	8	0	0x20
000000	10001	10010	01000	00000	100000

00000010001100100100000000100000 0x02324020

Chuyển đổi lệnh ASM MIPS sang mã máy

- Chuyển câu lệnh assembly (ASM) MIPS sau sang mã máy:
`and $t3, $s0, $s2`
- Thực hiện việc chuyển đổi theo các bước sau:
 - Bước 1: Tra bảng “MIPS reference data” xem lệnh *and* thuộc định dạng nào => R type

And and **R** $R[rd] = R[rs] \& R[rt]$ 0 / 24_{hex}

op	rs	rt	rd	shamt	funct

Bảng “MIPS reference data”

Information to separate card 2. Fold bottom side (columns 3 and 4) together

MIPS Reference Data

①



CORE INSTRUCTION SET

NAME, MNEMONIC	FOR-MAT	OPERATION (in Verilog)	OPCODE / FUNCT (Hex)
Add	add	R R[rd] = R[rs] + R[rt]	(1) 0 / 20 _{hex}
Add Immediate	addi	I R[rt] = R[rs] + SignExtImm	(1,2) 8 _{hex}
Add Imm. Unsigned	addiu	I R[rt] = R[rs] + SignExtImm	(2) 9 _{hex}
Add Unsigned	addu	R R[rd] = R[rs] + R[rt]	0 / 21 _{hex}
And	and	R R[rd] = R[rs] & R[rt]	0 / 24 _{hex}
And Immediate	andi	I R[rt] = R[rs] & ZeroExtImm	(3) c _{hex}
Branch On Equal	beq	I if(R[rs]==R[rt]) PC=PC+4+BranchAddr	(4) 4 _{hex}
Branch On Not Equal	bne	I if(R[rs]!=R[rt]) PC=PC+4+BranchAddr	(4) 5 _{hex}
Jump	j	J PC=JumpAddr	(5) 2 _{hex}
Jump And Link	jal	J R[31]=PC+8;PC=JumpAddr	(5) 3 _{hex}
Jump Register	jr	R PC=R[rs]	0 / 08 _{hex}
Load Byte Unsigned	lbu	I R[rt]={24'b0,M[R[rs] +SignExtImm](7:0)}	(2) 24 _{hex}
Load Halfword Unsigned	lhu	I R[rt]={16'b0,M[R[rs] +SignExtImm](15:0)}	(2) 25 _{hex}
Load Linked	ll	I R[rt] = M[R[rs]+SignExtImm]	(2,7) 30 _{hex}
Load Upper Imm.	lui	I R[rt] = {imm, 16'b0}	f _{hex}
Load Word	lw	I R[rt] = M[R[rs]+SignExtImm]	(2) 23 _{hex}
Nor	nor	R R[rd] = ~ (R[rs] R[rt])	0 / 27 _{hex}
Or	or	R R[rd] = R[rs] R[rt]	0 / 25 _{hex}
Or Immediate	ori	I R[rt] = R[rs] ZeroExtImm	(3) d _{hex}
Set Less Than	slt	R R[rd] = (R[rs] < R[rt]) ? 1 : 0	0 / 2a _{hex}

ARITHMETIC CORE INSTRUCTION SET

②

OPCODE / FMT / FT / FUNCT (Hex)

NAME, MNEMONIC	FOR-MAT	OPERATION	OPCODE / FMT / FT / FUNCT (Hex)
Branch On FP True	bclt	FI if(FPcond)PC=PC+4+BranchAddr	(4) 11/8/1/--
Branch On FP False	bclf	FI if(!FPcond)PC=PC+4+BranchAddr	(4) 11/8/0/--
Divide	div	R Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]	0/--/--/1a
Divide Unsigned	divu	R Lo=R[rs]/R[rt]; Hi=R[rs]%R[rt]	(6) 0/--/--/1b
FP Add Single	add.s	FR F[fd] = F[fs] + F[ft]	11/10/--/0
FP Add Double	add.d	FR {F[fd],F[fd+1]} = {F[fs],F[fs+1]} + {F[ft],F[ft+1]}	11/11/--/0
FP Compare Single	c.x.s*	FR FPcond = (F[fs] op F[ft]) ? 1 : 0	11/10/--/y
FP Compare Double	c.x.d*	FR FPcond = ({F[fs],F[fs+1]} op {F[ft],F[ft+1]}) ? 1 : 0	11/11/--/y
* (x is eq, lt, or le) (op is ==, <, or <=) (y is 32, 3c, or 3e)			
FP Divide Single	div.s	FR F[fd] = F[fs] / F[ft]	11/10/--/3
FP Divide Double	div.d	FR {F[fd],F[fd+1]} = {F[fs],F[fs+1]} / {F[ft],F[ft+1]}	11/11/--/3
FP Multiply Single	mul.s	FR F[fd] = F[fs] * F[ft]	11/10/--/2
FP Multiply Double	mul.d	FR {F[fd],F[fd+1]} = {F[fs],F[fs+1]} * {F[ft],F[ft+1]}	11/11/--/2
FP Subtract Single	sub.s	FR F[fd]=F[fs] - F[ft]	11/10/--/1
FP Subtract Double	sub.d	FR {F[fd],F[fd+1]} = {F[fs],F[fs+1]} - {F[ft],F[ft+1]}	11/11/--/1
Load FP Single	lwc1	I F[rt]=M[R[rs]+SignExtImm]	(2) 31/--/--/0
Load FP Double	ldc1	I F[rt]=M[R[rs]+SignExtImm]; F[rt+1]=M[R[rs]+SignExtImm+4]	(2) 35/--/--/0
Move From Hi	mfhi	R R[rd] = Hi	0/--/--/10
Move From Lo	mflo	R R[rd] = Lo	0/--/--/12
Move From Control	mfc0	R R[rd] = CR[rs]	10/0/--/0
Multiply	mult	R {Hi,Lo} = R[rs] * R[rt]	0/--/--/18
Multiply Unsigned	multu	R {Hi,Lo} = R[rs] * R[rt]	(6) 0/--/--/19
Shift Right Arith.	sra	R R[rd] = R[rt] >>> shamt	0/--/--/3
Store FP Single	swc1	I M[R[rs]+SignExtImm] = F[rt]	(2) 39/--/--/0
Store FP Double	sdc1	I M[R[rs]+SignExtImm] = F[rt]; M[R[rs]+SignExtImm+4] = F[rt+1]	(2) 3d/--/--/0

Bảng “MIPS reference data”

. Fold bottom side (columns 3 and 4) together

MIPS Reference Data

①



CORE INSTRUCTION SET

NAME, MNEMONIC	FOR-MAT	OPERATION (in Verilog)	OPCODE / FUNCT (Hex)
Add add	R	$R[rd] = R[rs] + R[rt]$	(1) $0 / 20_{hex}$
Add Immediate addi	I	$R[rt] = R[rs] + \text{SignExtImm}$	(1,2) 8_{hex}
Add Imm. Unsigned addiu	I	$R[rt] = R[rs] + \text{SignExtImm}$	(2) 9_{hex}
Add Unsigned addu	R	$R[rd] = R[rs] + R[rt]$	$0 / 21_{hex}$
And and	R	$R[rd] = R[rs] \& R[rt]$	$0 / 24_{hex}$
And Immediate andi	I	$R[rt] = R[rs] \& \text{ZeroExtImm}$	(3) c_{hex}
Branch On Equal beq	I	if($R[rs] == R[rt]$) $PC = PC + 4 + \text{BranchAddr}$	(4) 4_{hex}
Branch On Not Equal bne	I	if($R[rs] != R[rt]$) $PC = PC + 4 + \text{BranchAddr}$	(4) 5_{hex}
Jump j	J	$PC = \text{JumpAddr}$	(5) 2_{hex}
Jump And Link jal	J	$R[31] = PC + 8; PC = \text{JumpAddr}$	(5) 3_{hex}
Jump Register jr	R	$PC = R[rs]$	$0 / 08_{hex}$

Chuyển đổi lệnh ASM MIPS sang mã máy (tt)

- Chuyển câu lệnh assembly (ASM) MIPS sau sang mã máy:
and \$t3, \$s0, \$s2
- Thực hiện việc chuyển đổi theo các bước sau:
 - Bước 1: Tra bảng “MIPS reference data” xem lệnh *and* thuộc định dạng nào => R type
 - Bước 2: Tra các trường opcode và function (nếu có)

And

and

R $R[rd] = R[rs] \& R[rt]$

0 / 24_{hex}

op	rs	rt	rd	shamt	funct
000000					100100

Chuyển đổi lệnh ASM MIPS sang mã máy (tt)

- Chuyển câu lệnh assembly (ASM) MIPS sau sang mã máy:

and \$t3, \$s0, \$s2 => \$t3 = \$s0 & \$s2

- Thực hiện việc chuyển đổi theo các bước sau:
 - Bước 1: Tra bảng “MIPS reference data” xem lệnh *and* thuộc định dạng nào => R type
 - Bước 2: Tra các trường opcode và function (nếu có)
 - Bước 3: Tra vị trí và chỉ số các thanh ghi

And	and	R R[rd] = R[rs] & R[rt]				0 / 24 _{hex}
op	rs	rt	rd	shamt	funct	
000000	10000	10010	01011		100100	

NAME	NUMBER
\$zero	0
\$at	1
\$v0-\$v1	2-3
\$a0-\$a3	4-7
\$t0-\$t7	8-15
\$s0-\$s7	16-23
\$t8-\$t9	24-25
\$k0-\$k1	26-27
\$gp	28
\$sp	29
\$fp	30
\$ra	31

Chuyển đổi lệnh ASM MIPS sang mã máy (tt)

- Chuyển câu lệnh assembly (ASM) MIPS sang mã máy:

and **\$t3**, **\$s0**, **\$s2** => **\$t3** = **\$s0** & **\$s2**

- Thực hiện việc chuyển đổi theo các bước sau:
 - Bước 1: Tra bảng “MIPS reference data” xem lệnh *and* thuộc định dạng nào => R type
 - Bước 2: Tra các trường opcode và function (nếu có)
 - Bước 3: Tra vị trí và chỉ số các thanh ghi
 - Bước 4: Điền trường shamt và hoàn thành mã máy của lệnh

op	rs	rt	rd	shamt	funct
000000	10000	10010	01011	00000	100100

00000010000100100101100000100100 ➡ 0x02125824

Định dạng lệnh - R_type - Ví dụ 2

op	rs	rt	rd	shamt	funct
6 bit	5 bit	5 bit	5 bit	5 bit	6 bit

sll \$t0, \$s2, 2

op (sll)	0	\$s2	\$t0	2	funct (sll)
0	0	18	8	2	0x0
000000	00000	10010	01000	00010	000000

0000000000001001001000000010000000 0x00124080

Định dạng lệnh - R_type - Bài tập

Biểu diễn các lệnh sau dưới dạng mã máy theo kiến trúc MIPS (hex):

- add \$a0, \$t1, \$sp
- srl \$t1, \$t5, 7

op	rs	rt	rd	shamt	funct
6 bit	5 bit	5 bit	5 bit	5 bit	6 bit

NAME	NUMBER
\$zero	0
\$at	1
\$v0-\$v1	2-3
\$a0-\$a3	4-7
\$t0-\$t7	8-15
\$s0-\$s7	16-23
\$t8-\$t9	24-25
\$k0-\$k1	26-27
\$gp	28
\$sp	29
\$fp	30
\$ra	31

7. Biểu diễn lệnh

I_type

Định dạng lệnh - I_type

Các trường lệnh dạng I:

- op (opcode): Mã lệnh
- rs: Địa chỉ toán hạng thanh ghi nguồn thứ nhất
- rt: Địa chỉ toán hạng thanh ghi nguồn thứ hai hoặc thanh ghi đích
- immediate: Số tức thời 16 bit (biểu diễn dạng bù 2)
 - Thường là độ dời
 - Quy ước: Nếu sử dụng số tức thời lớn hơn 16 bit thì sẽ gây lỗi biên dịch

op	rs	rt	immediate
6 bit	5 bit	5 bit	16 bit

Định dạng lệnh - I_type - Ví dụ 1

op	rs	rt	immediate
6 bit	5 bit	5 bit	16 bit

lw \$t0, -8(\$s2)

op (lw)	\$s2	\$t0	-8
0x23	18	8	-8
100011	10010	01000	11111111111111000

100011100100100011111111111111000 0x4E48FFF8

Định dạng lệnh - I_type - Ví dụ 2

op	rs	rt	immediate
6 bit	5 bit	5 bit	16 bit

addi \$t0, \$s2, -8

op (addi)	\$s2	\$t0	-8
0x8	18	8	-8
001000	10010	01000	1111111111111000

00100010010010001111111111111000 0x2248FFF8

Định dạng lệnh - I_type - Ví dụ 3

op	rs	rt	immediate
6 bit	5 bit	5 bit	16 bit

slti \$t0, \$s2, -8

op (slti)	\$s2	\$t0	-8
0xA	18	8	-8
001010	10010	01000	11111111111111000

001010100100100011111111111111000 0x2A48FFF8

Định dạng lệnh - I_type - Ví dụ 4

op	rs	rt	immediate
6 bit	5 bit	5 bit	16 bit

0x400000: beq \$s2, \$t0, PASS
0x400004: slti \$t0, \$s2, 8
0x400008: add \$t0, \$s1, \$s2
0x40000C: PASS:

op (beq)	\$s2	\$t0	PASS = PC + 4 + {extS(imm) << 2}
000100	10010	01000	000000000000000010

00010010010010000000000000000010 0x12480002

Định dạng lệnh - I_type - Bài tập

Biểu diễn các lệnh sau dưới dạng mã máy theo kiến trúc MIPS (hex):

- lw \$a0, 48(\$sp)
- sw \$s0, -24(\$t5)

op	rs	rt	immediate
6 bit	5 bit	5 bit	16 bit

NAME	NUMBER
\$zero	0
\$at	1
\$v0-\$v1	2-3
\$a0-\$a3	4-7
\$t0-\$t7	8-15
\$s0-\$s7	16-23
\$t8-\$t9	24-25
\$k0-\$k1	26-27
\$gp	28
\$sp	29
\$fp	30
\$ra	31

7. Biểu diễn lệnh

J_type

Định dạng lệnh - J_type

Các trường lệnh dạng J:

- op (opcode): Mã lệnh
- address: Số tức thời 26 bit (biểu diễn dạng bù 2)
 - Bit [27:2] của địa chỉ nhảy tới

op	address
6 bit	26 bit

Định dạng lệnh - J_type - Ví dụ 1

op	address
6 bit	26 bit
	0xCAFEBAB8: j sub_pro
	...
	0xCAFEBAFC: sub_pro:
	1100 1010 1111 1110 1011 1010 1111 1100
op (j)	0xCAFEBAFC = {(PC+4)[31:28], address, 2'b0}
0x2	0x2BFAEBF
000010	10101111111010111010111111

0000101010111111111010111010111111 0x0ABFAEBF

Định dạng lệnh - J_type - Bài tập

Biểu diễn các lệnh sau dưới dạng mã máy theo kiến trúc MIPS (hex):

- j IT012
 - Lệnh j ở địa chỉ 0x00CAFE00
 - Nhãn IT012 có địa chỉ tương ứng là 0x0000A5B0

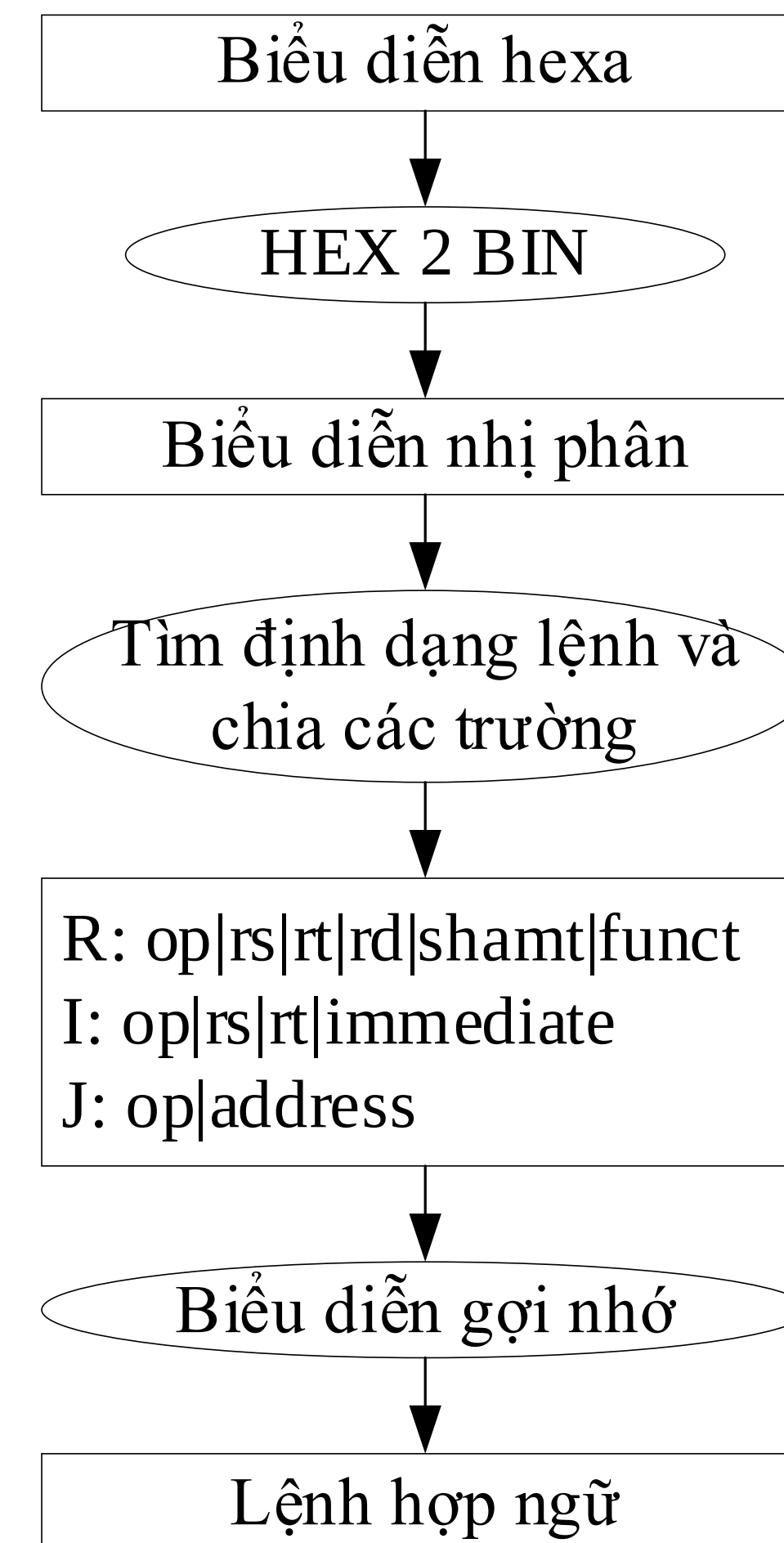
op	address
6 bit	26 bit

7. Biểu diễn lệnh

Biên dịch ngược

Biên dịch ngược

- Biên dịch ngược là quá trình khôi phục mã máy thành chương trình hợp ngữ
- 0x00af8020
- 0000 0000 1010 1111 1000 0000 0010 0000
- 000000 00101 01111 10000 00000 100000
- add \$16, \$5, \$15 hoặc add \$s0, \$a1, \$t7



Chuyển mã máy sang ngôn ngữ Assembly (ASM) MIPS

- Chuyển mã máy sau sang câu lệnh assembly MIPS:

0x01304024
- Thực hiện việc chuyển đổi theo các bước sau:
 - Bước 1: Chuyển mã máy sang hệ nhị phân
 - Bước 2: Chọn 6bit đầu và tra opcode, nếu opcode bằng 000000 thì tra tiếp function 6bit cuối trong bảng MIPS

0x	0	1	3	0	4	0	2	4
	0000	0001	0011	0000	0100	0000	0010	0100

op		funct
000000	01 0011 0000 0100 0000 00	100100

CORE INSTRUCTION SET				OPCODE / FUNCT (Hex)
NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)	
Add	add	R	R[rd] = R[rs] + R[rt]	(1) 0 / 20 _{hex}
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2) 8 _{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2) 9 _{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]	0 / 21 _{hex}
And	and	R	R[rd] = R[rs] & R[rt]	0 / 24 _{hex}

Từ đó suy ra lệnh “and” và thuộc format “R”

Chuyển mã máy sang ngôn ngữ Assembly MIPS (tt)

- Chuyển mã máy sau sang câu lệnh assembly MIPS:

0x01304024
- Thực hiện việc chuyển đổi theo các bước sau:
 - Bước 2: Chọn 6bit đầu và tra opcode, nếu opcode bằng 000000 thì tra tiếp function 6bit cuối trong bảng MIPS
 - Bước 3: Tìm các thanh ghi tương ứng với các trường trong format R

op						funct
000000	01 0011 0000 0100 0000 00					100100

op	Rs	rt	rd	shamt	funct
000000	01001	10000	01000	00000	100100
	9 ⇒ t1	16 ⇒ s0	8 ⇒ t0		

CORE INSTRUCTION SET

NAME, MNEMONIC	FOR- MAT	OPERATION (in Verilog)	OPCODE / FUNCT (Hex)
Add	add	R R[rd] = R[rs] + R[rt]	(1) 0 / 20 _{hex}
Add Immediate	addi	I R[rt] = R[rs] + SignExtImm	(1,2) 8 _{hex}
Add Imm. Unsigned	addiu	I R[rt] = R[rs] + SignExtImm	(2) 9 _{hex}
Add Unsigned	addu	R R[rd] = R[rs] + R[rt]	0 / 21 _{hex}
And	and	R R[rd] = R[rs] & R[rt]	0 / 24 _{hex}
And Immediate	andi	I R[rt] = R[rs] & ZeroExtImm	(3) c _{hex}

Chuyển mã máy sang ngôn ngữ Assembly MIPS (tt)

- Chuyển mã máy sau sang câu lệnh assembly MIPS:

0x01304024
- Thực hiện việc chuyển đổi theo các bước sau:
 - Bước 2: Chọn 6bit đầu và tra opcode, nếu opcode bằng 000000 thì tra tiếp function 6bit cuối trong bảng MIPS
 - Bước 3: Tìm các thanh ghi tương ứng với các trường trong format R
 - Bước 4: Hoàn thành lệnh: and \$t0, \$t1, \$s0

op	Rs	rt	rd	shamt	funct
000000	01001	10000	01000	00000	100100
	9 ⇒ t1	16 ⇒ s0	8 ⇒ t0		

CORE INSTRUCTION SET

NAME, MNEMONIC	FOR- MAT	OPERATION (in Verilog)	OPCODE / FUNCT (Hex)
Add	R	R[rd] = R[rs] + R[rt]	(1) 0 / 20 _{hex}
Add Immediate	I	R[rt] = R[rs] + SignExtImm	(1,2) 8 _{hex}
Add Imm. Unsigned	I	R[rt] = R[rs] + SignExtImm	(2) 9 _{hex}
Add Unsigned	R	R[rd] = R[rs] + R[rt]	0 / 21 _{hex}
And	R	R[rd] = R[rs] & R[rt]	0 / 24 _{hex}
And Immediate	I	R[rt] = R[rs] & ZeroExtImm	(3) c _{hex}

Biên dịch ngược - Bài tập

- Chuyển mã máy sau sang câu lệnh assembly MIPS:

- `0x2128fff3`

- `0xad28fffc`



8. Các lệnh điều kiện và nhảy

Nhóm lệnh số học và luận lý

8. Các lệnh điều kiện và nhảy

- Một máy tính (PC) khác với các máy tính tay (calculator) chính là dựa trên khả năng đưa ra quyết định.
- Trong ngôn ngữ lập trình, đưa ra quyết định thường được biểu diễn bằng cách sử dụng câu lệnh “if”, đôi khi kết hợp với câu lệnh “go to”.
- Ngôn ngữ Assembly MIPS cũng chứa các lệnh hỗ trợ ra quyết định, tương tự với câu lệnh “if” và “go to”.
 - Ví dụ: `beq register1, register2, L1`
 - Lệnh này có nghĩa là đi đến câu lệnh có nhãn L1 nếu giá trị của thanh ghi register1 bằng giá trị thanh ghi register2.
- Từ ‘beq’ là viết tắt của “branch if equal” (rẽ nhánh nếu bằng)
- Các lệnh như ‘beq’ được gọi là lệnh rẽ nhánh có điều kiện.

8. Các lệnh điều kiện và nhảy (tt)

- Các lệnh rẽ nhánh có điều kiện (conditional branch) của MIPS:

Conditional branch	branch on equal	<code>beq \$s1,\$s2,25</code>	if ($\$s1 == \$s2$) go to $PC + 4 + 100$
	branch on not equal	<code>bne \$s1,\$s2,25</code>	if ($\$s1 \neq \$s2$) go to $PC + 4 + 100$
	set on less than	<code>slt \$s1,\$s2,\$s3</code>	if ($\$s2 < \$s3$) $\$s1 = 1$; else $\$s1 = 0$
	set on less than unsigned	<code>sltu \$s1,\$s2,\$s3</code>	if ($\$s2 < \$s3$) $\$s1 = 1$; else $\$s1 = 0$
	set less than immediate	<code>slti \$s1,\$s2,20</code>	if ($\$s2 < 20$) $\$s1 = 1$; else $\$s1 = 0$
	set less than immediate unsigned	<code>sltiu \$s1,\$s2,20</code>	if ($\$s2 < 20$) $\$s1 = 1$; else $\$s1 = 0$

8. Các lệnh điều kiện và nhảy - “ $\geq$ ”

- Nếu ta muốn so sánh if ($s0 \geq s1$) goto ABC thì ta dùng cặp slt và beq

slt \$t0, \$s0, \$s1

\$t0 = 1 if \$s0 < \$s1

beq \$t0, \$zero, ABC

if \$s0 $\geq$ \$s1, goto ABC

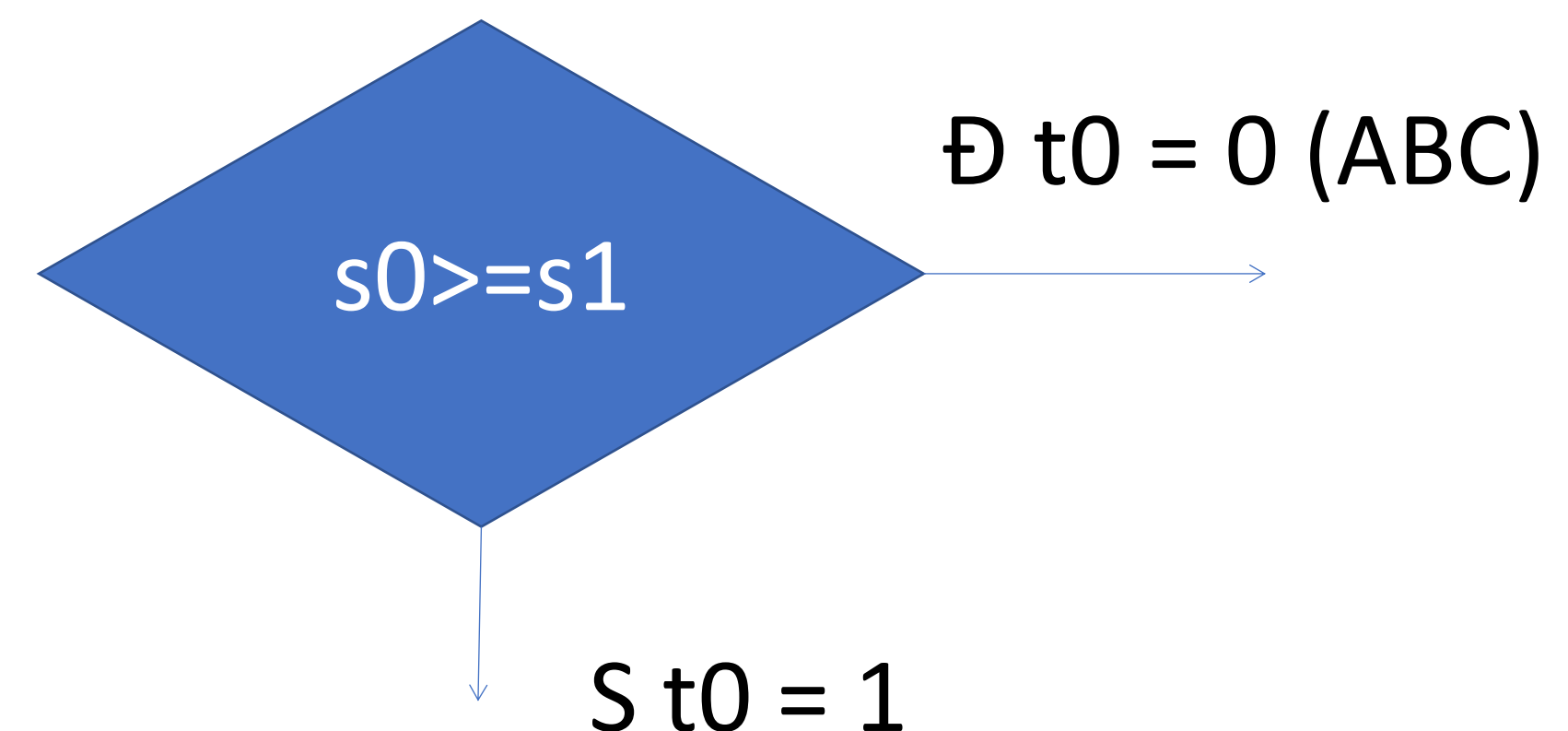
<stuff>

do if \$s0 < \$s1

j Exit

ABC:

Exit:



8. Các lệnh điều kiện và nhảy - “<”

- Nếu ta muốn so sánh if ($s0 < s1$) goto ABC thì ta dùng cặp `slt` và `bne`

```
slt $t0, $s0, $s1
```

```
# $t0 = 1 if $s0 < $s1
```

```
bne $t0, $zero, ABC
```

```
# if $s0 < $s1, goto ABC
```

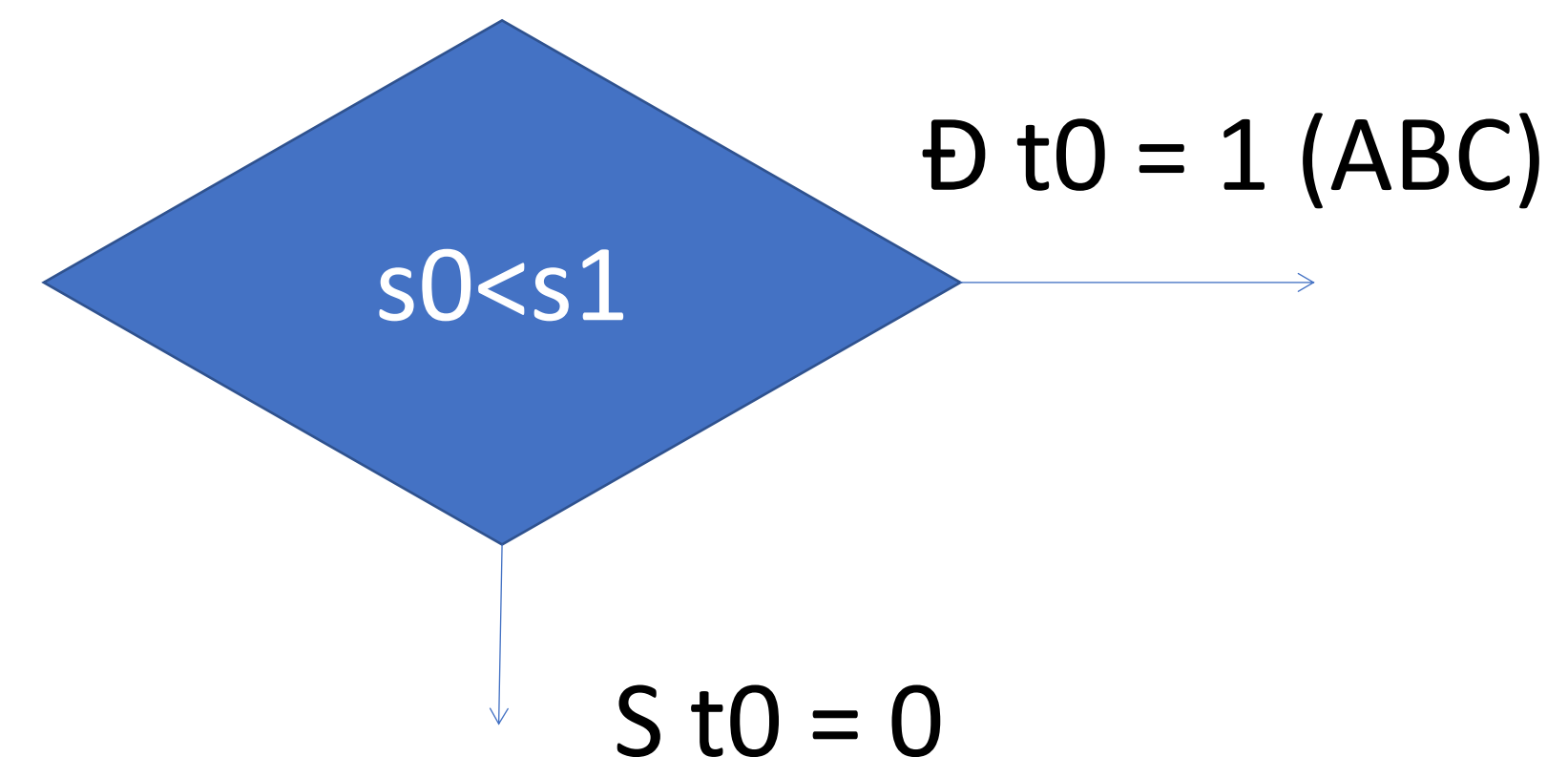
```
<stuff>
```

```
# do if $s0 >= $s1
```

```
j Exit
```

```
ABC: ....
```

```
Exit:
```



8. Các lệnh điều kiện và nhảy (tt)

- Ngoài ra còn có các lệnh rẽ nhánh có điều kiện khác, nhưng là nhóm lệnh giả (pseudo instructions):

Conditional branch (pseudo instruction)	branch on less than	blt
	branch greater than	bgt
	branch less than or equal	ble
	branch greater than or equal	bge

8. Các lệnh điều kiện và nhảy (tt)

- Các lệnh rẽ nhánh không điều kiện (unconditional branch) của MIPS:

Unconditional jump	jump	j label
	jump register	jr \$ra
	jump and link	jal label

8. Các lệnh điều kiện và nhảy - Ví dụ 1

- Biên dịch if-then-else từ ngôn ngữ cấp cao sang assembly MIPS cho đoạn mã sau:

if (i == j) f = g + h; else f = g - h;

- Biết f, g, h, i và j là các biến. Nếu năm biến f đến j tương ứng với 5 thanh ghi \$s0 đến \$s4, mã MIPS cho câu lệnh if này là gì?

Trả lời:

bne \$s3, \$s4, Else	# go to Else if i != j
add \$s0, \$s1, \$s2	# f = g + h (skipped if i != j)
j exit	# go to Exit
Else: sub \$s0, \$s1, \$s2	# f = g - h (skipped if i = j)
exit:	

8. Các lệnh điều kiện và nhảy - Ví dụ 2

- Biên dịch 1 vòng lặp while từ ngôn ngữ cấp cao sang assembly MIPS cho đoạn mã sau:
while (save[i] == k)
 i += 1;
- Giả định rằng i và k tương ứng với thanh ghi \$s3 và \$s5; và địa chỉ nền/cơ sở của mảng save lưu trong \$s6. Mã assembly MIPS tương ứng với đoạn mã C trên là gì?

Trả lời:

Loop: sll \$t1, \$s3, 2	# Temp reg \$t1 = 4 * i
add \$t1, \$t1, \$s6	# \$t1 = address of save[i]
lw \$t0, 0(\$t1)	# Temp reg \$t0 = save[i]
bne \$t0, \$s5, Exit	# go to Exit if save[i] != k
addi \$s3, \$s3, 1	# i = i + 1
j Loop	# go to Loop
Exit:	

8. Các lệnh điều kiện và nhảy - Ví dụ 3

- Biên dịch đoạn lệnh if từ ngôn ngữ cấp cao sang assembly MIPS. Cho đoạn mã sau:

```
if (i > j)
    { A[i] = A[3] + 1;}
else
    { A[i+1] = 10;}
i++;
```

- Biết i và j tương ứng với các thanh ghi \$s0 và \$s1. Mảng A là mảng mà các phần tử là số nguyên, mỗi phần tử chiếm 1 từ nhớ (4 bytes) và địa chỉ nền của mảng A lưu trong thanh ghi \$s3

8. Các lệnh điều kiện và nhảy - Ví dụ 3

if ($i > j$)

{ $A[i] = A[3] + 1;$ }

else

{ $A[i+1] = 10;$ }

$i++;$



Exit: *addi \$s0 \$s0, 1*

→ *slt \$t0, \$s1, \$s0*
bne \$t0, \$zero, if hoặc *beq \$t0, \$zero, else*

if: *lw \$t1, 12 (\$s3)*
addi \$t1, \$t1, 1
sll \$t2, \$s0, 2
add \$t2, \$t2, \$s3
sw \$t1, 0 (\$t2)
j Exit

else: *addi \$t3, \$zero, 10*
addi \$t4, \$s0, 1
sll \$t4, \$t4, 2
add \$t4, \$t4, \$s3
sw \$t3, 0 (\$t4)
j Exit

- Biết i và j tương ứng với các thanh ghi $\$s0$ và $\$s1$. Mảng A là mảng mà các phần tử là số nguyên, mỗi phần tử chiếm 1 từ nhớ (4 bytes) và địa chỉ nền của mảng A lưu trong thanh ghi $\$s3$

8. Các lệnh điều kiện và nhảy - Ví dụ 3

- Chương trình sau khi ghép lại với nhau:

```
slt $t0, $s1, $s0
```

```
bne $t0, $zero, if
```

```
else: addi $t3, $zero, 10
```

```
    addi $t4, $s0, 1
```

```
    sll $t4, $t4, 2
```

```
    add $t4, $t4, $s3
```

```
    sw $t3, 0 ($t4)
```

```
    j Exit
```

```
if: lw $t1, 12 ($s3)
```

```
    addi $t1, $t1, 1
```

```
    sll $t2, $s0, 2
```

```
    add $t2, $t2, $s3
```

```
    sw $t1, 0 ($t2)
```

```
    j Exit
```

```
Exit: addi $s0 $s0, 1
```

8. Các lệnh điều kiện và nhảy - Bài tập

- Biên dịch 1 vòng lặp for từ ngôn ngữ cấp cao sang assembly MIPS cho đoạn mã sau:
 j = value;
 for(i = 1; i < j; i++)
 A[i] = B[i];
 j = 0;
- Biết i và j tương ứng với các thanh ghi \$s0 và \$s1. Mảng A là mảng mà các phần tử là số nguyên, mỗi phần tử chiếm 1 từ nhớ (4 bytes) và địa chỉ nền của mảng A lưu trong thanh ghi \$s3. Với địa chỉ nền mảng B đang lưu trong thanh ghi \$s4 và biến value tương ứng thanh ghi \$s5

Tóm tắt lại nội dung buổi học

- Biểu diễn lệnh
 - Các định dạng lệnh
 - Chuyển lệnh ASM sang mã máy
 - Chuyển lệnh từ mã máy sang ASM
- Các lệnh điều kiện và nhảy
 - Các lệnh nhảy có điều kiện
 - Các lệnh giả
 - Các lệnh nhảy không điều kiện



THẢO LUẬN

